Binary Logistic Regression Case Study: ICU Mortality Risk Factor Model

Background:

This Data is for a Study of Risk Factors Associated with ICU Mortality. Data was collected at Baystate Medicla Center, Springfield, Massachusetts.

Source :

APPLIED LOGISTIC REGRESSION by DAVID W. HOSMER & STANLEY LEMESHOW pg no.253.

Objective :

The objective is to investigate the relationship between various predictor variables listed in ICU Mortality data and the binary outcome variable "Vital Status" (STA), which indicates whether a patient lived (0) or died (1) during their stay in the Intensive Care Unit (ICU) at Baystate Medical Center, Springfield, Massachusetts. This analysis aims to identify and quantify the risk factors associated with ICU mortality based on the provided dataset

Import Libraries

In [277]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf

from sklearn.metrics import classification_report
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import roc_curve, roc_auc_score
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.metrics import roc_curve, roc_auc_score

import warnings
warnings.filterwarnings("ignore")
```

Load Data

In [278]:
```
data = pd.read_csv("ICU Mortality.csv")
data.rename(columns = {'INF':'INF_C'},inplace = True)
data.head()
```

Out[278]:

	SR NO	ID	STA	AGE	SEX	RACE	SER	CAN	CRN	INF_C	...	HRA	PRE	TYP	FRA	PO2	P
0	1	8	0	27	1	1	0	0	0	1	...	88	0	1	0	0	
1	2	12	0	59	0	1	0	0	0	0	...	80	1	1	0	0	
2	3	14	0	77	0	1	1	0	0	0	...	70	0	0	0	0	
3	4	28	0	54	0	1	0	0	0	1	...	103	0	1	1	0	
4	5	32	0	87	1	1	1	0	0	1	...	154	1	1	0	0	

5 rows × 22 columns

Data Description

Column	Description	Values
ID	Identification Code	
STA	Vital Status	0 = Lived, 1 = Died
AGE	Age in Years	
SEX	Sex	0 = Male, 1 = Female
RACE	Race	1 = White, 2 = Black, 3 = Other
SER	Service at ICU Admission	0 = Medical, 1 = Surgical
CAN	Cancer Part of Present Problem	0 = No, 1 = Yes
CRN	History of Chronic Renal Failure	0 = No, 1 = Yes
INF	Infection Probable at ICU Admission	0 = No, 1 = Yes
CPR	CPR Prior to ICU Admission	0 = No, 1 = Yes
SYS	Systolic Blood Pressure at ICU Admission	mm Hg
HRA	Heart Rate at ICU Admission	Beats/Minute
PRE	Previous Admission to an ICU within 6 Months	0 = No, 1 = Yes
TYP	Type of Admission	0 = Elective, 1 = Emergency
FRA	Long Bone, Multiple, Neck, Single Area, or Hip Fracture	0 = No, 1 = Yes
PO2	PO2 from Initial Blood Gases	0 = > 60, 1 = ≤ 60
PH	PH from Initial Blood Gases	0 = ≥ 7.25, 1 = < 7.25
PCO	PCO2 from Initial Blood Gases	0 = ≤ 45, 1 = > 45
BIC	Bicarbonate from Initial Blood Gases	0 = ≥ 18, 1 = < 18
CRE	Creatinine from Initial Blood Gases	0 = ≤ 2.0, 1 = > 2.0
LOC	Level of Consciousness at ICU Admission	0 = No Coma or Stupor, 1 = Deep Stupor, 2 = Coma

Vital Status Statistics

In [279]:
```
df = data.copy()
df['STA'].replace({0: 'Lived', 1: 'Dead'}, inplace=True)
#To get the count
count = df['STA'].value_counts().reset_index()
#To get percentage count
per = df['STA'].value_counts(normalize = True).reset_index()
df = pd.merge(count,per,on = ['index'])
df.columns = ['Vital Status','Count','Proportion']
df['Proportion'] = df['Proportion']*100

df
```

Out[279]:

	Vital Status	Count	Proportion
0	Lived	160	80.0
1	Dead	40	20.0

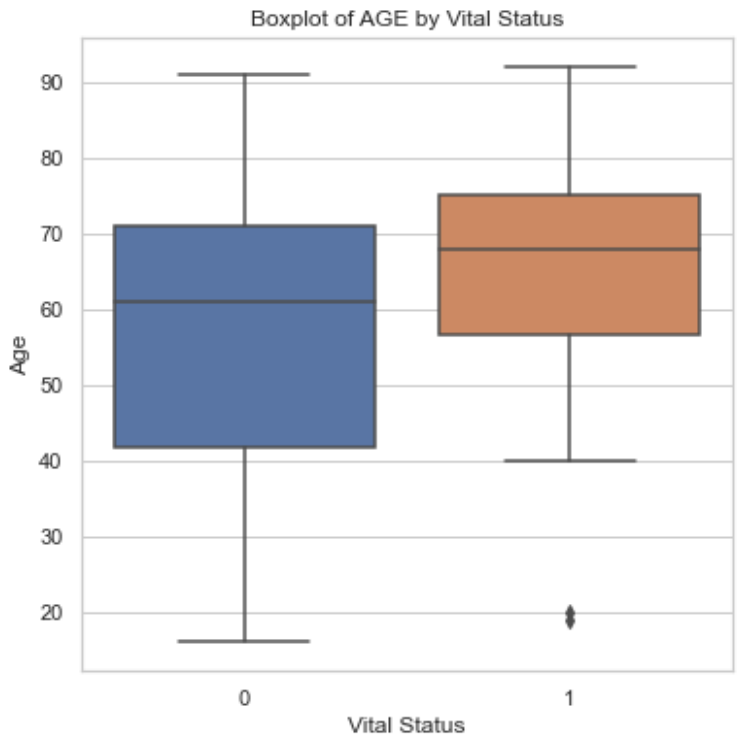
Observations :

The table shows that one out of five patients admitted to the ICU did not survive

Boxplot of AGE by Vital Status

In [280]:
```
# Set the style for the plot (optional)
sns.set(style="whitegrid")

# Create a boxplot using Seaborn
plt.figure(figsize=(6, 6))
sns.boxplot(x='STA', y='AGE', data=data)
plt.title('Boxplot of AGE by Vital Status')
plt.xlabel('Vital Status')
plt.ylabel('Age')
plt.show()
```



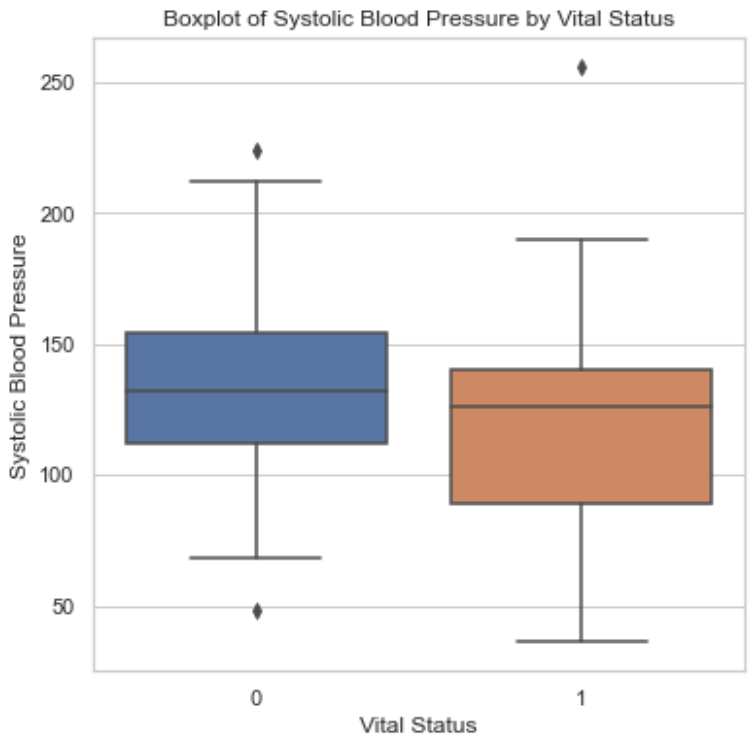
Observations :

Box-Whisker plot indicates that higher age is associated with ICU mortality

Boxplot of Systolic Blood Pressure by Vital Status

In [281]:
```
# Set the style for the plot (optional)
sns.set(style="whitegrid")

# Create a boxplot using Seaborn
plt.figure(figsize=(6, 6))
sns.boxplot(x='STA', y='SYS', data=data)
plt.title('Boxplot of Systolic Blood Pressure by Vital Status')
plt.xlabel('Vital Status')
plt.ylabel('Systolic Blood Pressure')
plt.show()
```



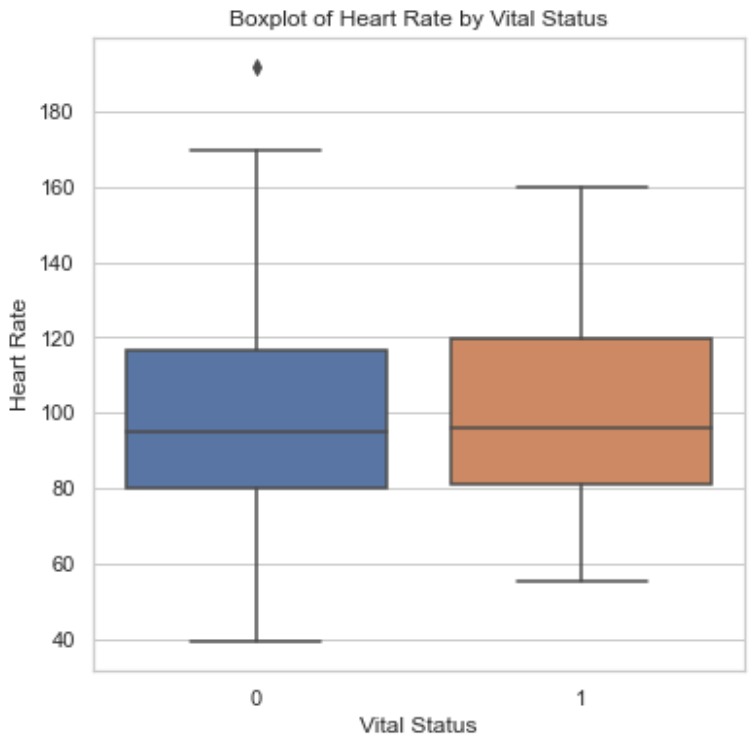
Observations :

Low blood pressure is associated with high mortality rate

Boxplot of Heart Rate by Vital Status

In [282]:
```
# Set the style for the plot (optional)
sns.set(style="whitegrid")

# Create a boxplot using Seaborn
plt.figure(figsize=(6, 6))
sns.boxplot(x='STA', y='HRA', data=data)
plt.title('Boxplot of Heart Rate by Vital Status')
plt.xlabel('Vital Status')
plt.ylabel('Heart Rate')
plt.show()
```



Observations :

Heart Rate (bases on median/quartiles) is not associated with ICU mortality

Sex - Based Mortality Statistics

In [283]:
```
# Group the data by SEX and calculate the count of STA for each group
df = data.copy()
df['SEX'].replace({0: 'Male', 1: 'Female'}, inplace=True)

grouped_sex = df.groupby('SEX')['STA'].agg(['count', 'sum']).reset_index()

# Rename the columns for clarity
grouped_sex.columns = ['SEX', 'Total Count', 'Death Count']

# Calculate the death rate as a percentage
grouped_sex['Death Rate(%)'] = (grouped_sex['Death Count'] / grouped_sex['Tota
l Count']) * 100
grouped_sex.round(2)
```

Out[283]:

	SEX	Total Count	Death Count	Death Rate(%)
0	Female	76	16	21.05
1	Male	124	24	19.35

Observations :

The table shows a gender imbalance, with more males (124) than females (76), and although more males experienced death (24), the death rate was slightly higher among females (21.05% vs. 19.35%).

Type Of Admission (TYP) - Based Mortality Statistics

In [284]:
```
df = data.copy()
df['TYP'].replace({0: 'Elective', 1: 'Emergency'}, inplace=True)

# Group the data by SEX and calculate the count of STA for each group
grouped_typ = df.groupby('TYP')['STA'].agg(['count', 'sum']).reset_index()

# Rename the columns for clarity
grouped_typ.columns = ['TYP', 'Total Count', 'Death Count']

# Calculate the death rate as a percentage
grouped_typ['Death Rate(%)'] = (grouped_typ['Death Count'] / grouped_typ['Tota
l Count']) * 100
grouped_typ.round(2)
```

Out[284]:

	TYP	Total Count	Death Count	Death Rate(%)
0	Elective	53	2	3.77
1	Emergency	147	38	25.85

Observation :

As expected patients admitted as emergency type shows higher ICU mortality rate as compared to elective admission.

Level Of Consciousness (LOC) - Based Mortality Statistics

In [285]:
```
df = data.copy()
df['LOC'].replace({0: 'No Coma Or Stupor', 1: 'Deep Stupor', 2: 'Coma'}, inpla
ce=True)

# Group the data by SEX and calculate the count of STA for each group
grouped_loc = df.groupby('LOC')['STA'].agg(['count', 'sum']).reset_index()

# Rename the columns for clarity
grouped_loc.columns = ['LOC', 'Total Count', 'Death Count']

# Calculate the death rate as a percentage
grouped_loc['Death Rate(%)'] = (grouped_loc['Death Count'] / grouped_loc['Tota
l Count']) * 100
grouped_loc.round(2)
```

Out[285]:

	LOC	Total Count	Death Count	Death Rate(%)
0	Coma	10	8	80.00
1	Deep Stupor	5	5	100.00
2	No Coma Or Stupor	185	27	14.59

Observations :

Level of consciousness at admission is clearly associated with ICU mortality. The mortality rate being highest for Deep Stupor followed by Coma and No Coma or Stupor

Logistic Regression Model

"LOC" values greater than 0, representing both "Deep Stupor" and "Coma," are combined and denoted as 1 due to their limited occurrence for simplified analysis.

In [286]:
```
data['LOC']  = np.where(data['LOC']>0,1,0)
```

We are excluding the variable "RACE" from the analysis due to its skewed distribution, with a predominant number of instances classified as "WHITE" and minimal representation for "Black" and other racial categories

In [287]:
```python
# Step 1: Define your formula
formula = 'STA ~ AGE + SEX + SER + CAN + CRN  + INF_C + CPR + SYS + HRA + PRE
+ TYP + FRA + PO2 + PH + PCO + BIC + CRE + LOC'

# Step 2: Fit the linear regression model
model = smf.logit(formula, data=data).fit()

# Step 3: Print the model summary
print(model.summary())
```

```
Warning: Maximum number of iterations has been exceeded.
         Current function value: 0.325995
         Iterations: 35
                           Logit Regression Results
==============================================================================
Dep. Variable:                    STA   No. Observations:                  200
Model:                          Logit   Df Residuals:                      181
Method:                           MLE   Df Model:                           18
Date:                Fri, 29 Sep 2023   Pseudo R-squ.:                  0.3485
Time:                        15:50:12   Log-Likelihood:                -65.199
converged:                       False   LL-Null:                       -100.08
Covariance Type:            nonrobust   LLR p-value:                 4.957e-08
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -3.7910      1.858     -2.040      0.041      -7.434      -0.148
AGE            0.0437      0.016      2.735      0.006       0.012       0.075
SEX           -0.3125      0.481     -0.649      0.516      -1.256       0.631
SER           -0.4591      0.571     -0.805      0.421      -1.578       0.659
CAN          -20.3059   4.28e+04     -0.000      1.000    -8.4e+04    8.39e+04
CRN            0.0171      0.770      0.022      0.982      -1.492       1.527
INF_C         -0.0083      0.534     -0.016      0.988      -1.055       1.039
CPR            1.0175      0.932      1.092      0.275      -0.809       2.844
SYS           -0.0141      0.008     -1.811      0.070      -0.029       0.001
HRA           -0.0031      0.009     -0.338      0.735      -0.021       0.015
PRE            0.6321      0.607      1.042      0.298      -0.557       1.821
TYP            1.9318      0.956      2.020      0.043       0.057       3.806
FRA            0.8281      0.988      0.838      0.402      -1.108       2.764
PO2            0.2138      0.776      0.275      0.783      -1.308       1.735
PH             2.0639      1.085      1.901      0.057      -0.063       4.191
PCO           -2.4211      1.180     -2.051      0.040      -4.735      -0.107
BIC           -0.8013      0.856     -0.936      0.349      -2.480       0.877
CRE            0.5011      0.933      0.537      0.591      -1.327       2.330
LOC            4.3062      1.081      3.984      0.000       2.188       6.425
==============================================================================
```

Observations :

The variables AGE(patient's age), LOC(level of consciousness), TYP(type of admission) and PCO (PCO2 from initial blood gases) are statistically significant (p <= 0.05) risk factors associated with ICU mortality. Further, the variable SYS (Systolic Blood Pressure) and PH (PH from initial blood gases) is statistically significant at 10% level of significance (p=0.07 and 0.057)

NOTE:

While our model includes several variables that are statistically insignificant, it is crucial to emphasize that in the healthcare domain, caution should be exercised when considering variable removal. In healthcare, even seemingly insignificant variables may hold clinical relevance or contribute to a comprehensive understanding of patient outcomes. Therefore, retaining these variables in the model allows for a more holistic analysis that aligns with the complex and multifaceted nature of healthcare data

Variance Inflation Factor (VIF) Analysis (checking Multicollinearity)

In [288]:
```
# Create a DataFrame to store VIF results
vif_data = pd.DataFrame()
vif_data["Feature"] = model.model.exog_names[1:]  # Exclude intercept
vif_data["VIF"] = [variance_inflation_factor(model.model.exog, i) for i in ran
ge(1, model.model.exog.shape[1])]

# Print the VIF DataFrame
print(vif_data)
```

```
   Feature       VIF
0      AGE  1.347618
1      SEX  1.096121
2      SER  1.927416
3      CAN  1.375821
4      CRN  1.319146
5    INF_C  1.332957
6      CPR  1.315199
7      SYS  1.171510
8      HRA  1.294332
9      PRE  1.167468
10     TYP  1.892265
11     FRA  1.259913
12     PO2  1.404664
13      PH  1.622939
14     PCO  1.744181
15     BIC  1.382282
16     CRE  1.260468
17     LOC  1.246863
```

Observations :

It is observed that no variable has high vif. Hence the problem of multicollinearity does not exist

Evaluate Model Performance by Checking Area Under The ROC Curve

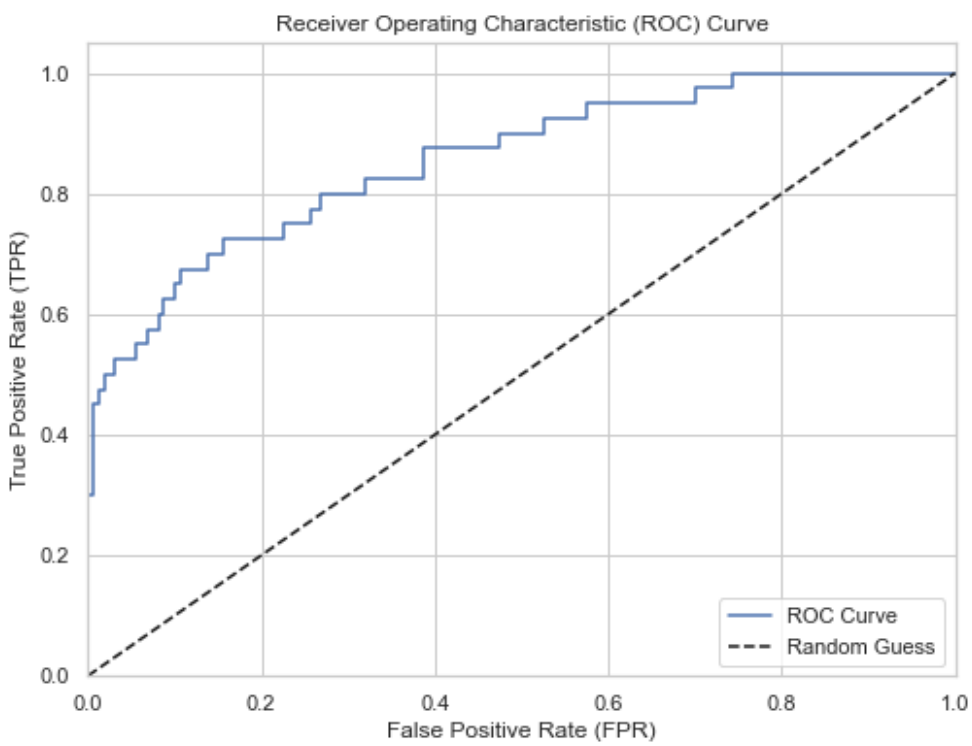
In [289]:
```
data['predicted_prob'] = model.predict(data)
# Step 1: Calculate ROC curve
fpr, tpr, thresholds = roc_curve(data['STA'], data['predicted_prob'])

# Step 2: Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label='ROC Curve')
plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')

# Step 3: Calculate AUC ROC
auc_roc = roc_auc_score(data['STA'], data['predicted_prob'])
print(f'AUC ROC: {auc_roc}')

# Step 4: Show the plot
plt.show()
```

```
AUC ROC: 0.85609375
```



Observations :

The model's AUC of 85.6% indicates a relatively strong discriminative ability, implying good predictive performance

Finding the Best Threshold

In [290]:
```
# Calculate Receiver Operating Characteristic (ROC) curve values.
fpr, tpr, thresholds = roc_curve(data['STA'], data['predicted_prob'])
# Find the optimal threshold index that maximizes the difference between true
positive rate (tpr) and false positive rate (fpr).
optimal_idx = np.argmax(tpr - fpr)

# Get the corresponding optimal threshold from the thresholds array.
optimal_threshold = round(thresholds[optimal_idx],3)

# Convert the optimal_threshold to a percentage string before printing
print(f'Best Threshold is : {optimal_threshold}')
```

```
Best Threshold is : 0.28
```

Classification Report

In [291]:
```
# Calculate predicted classes (usually a threshold of 0.5 is used for binary c
lassification)
data['predicted_class'] = (data['predicted_prob'] > optimal_threshold).astype
(int)

# Create the classification report
report = classification_report(data['STA'], data['predicted_class'])

# Print the classification report
print(report)
```

```
              precision    recall  f1-score   support

           0       0.91      0.89      0.90       160
           1       0.60      0.65      0.63        40

    accuracy                           0.84       200
   macro avg       0.76      0.77      0.76       200
weighted avg       0.85      0.84      0.85       200
```

Observations :

The sensitivity and specificity values using optimum threshold are approximately 65% and 89% and the accuracy is approximately 84% indicating good model performance.